LAPORAN PRAKTIKUM

DESAIN WEB

MODUL 2

Dosen Pengampu: Dede iraawan, M.kom

Disusun oleh :

Nama	**:**	**Naufal Nurrochmat Fadhil Haq**
NIM	**:**	**20250910036**
Kelas	**:**	**SINFC-02 2025**

SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS KUNINGAN

2026

KATA PENGANTAR

Laporan ini disusun untuk memenuhi tugas mata kuliah Praktikum Desain Web pada Program Studi Sistem Informasi, Fakultas Ilmu Komputer, Universitas Kuningan. Di dalam modul kedua ini, pembahasan difokuskan pada pengenalan konsep version control dan Git, pembuatan akun GitHub, instalasi dan konfigurasi Git, hingga penggunaan perintah dasar Git untuk mengelola proyek dan mengirimkannya ke remote repository di GitHub.

Saya menyadari bahwa penyusunan laporan ini tidak lepas dari dukungan berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta rekan-rekan mahasiswa yang telah memberikan arahan dan bimbingan selama sesi praktikum berlangsung.
Semoga laporan ini dapat memberikan manfaat, baik sebagai dokumentasi hasil belajar maupun sebagai referensi tambahan bagi pembaca yang sedang mempelajari Git dan GitHub dalam pengembangan web.

Kuningan, 29 Oktober 2026

DAFTAR ISI

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I 4
PRETEST **4**
BAB II 5
PEMBAHASAN **5**
BAB III 8
POSTEST **8**
BAB IV 9
TUGAS **9**
BAB V 11
KESIMPULAN **11**

BAB I

PRETEST

1. Apa tujuan dari menggunakan version control dalam pengembangan perangkat lunak?
2. Jelaskan perbedaan antara git add dan git commit!
3. Bagaimana cara membuat branch baru di Git?

Jawaban:

1. Version control berfungsi mencatat setiap perubahan pada kode program sehingga pengembangan menjadi lebih terkendali. Tujuan utamanya adalah:
   - **Menyimpan riwayat perubahan**, yaitu siapa yang mengubah, kapan, dan apa saja yang diubah.
   - **Mengembalikan versi lama** apabila perubahan terbaru menimbulkan kesalahan.
   - **Mendukung kerja kolaborasi**, karena banyak orang dapat mengerjakan proyek yang sama tanpa saling menimpa file.
   - **Memisahkan pengembangan fitur baru** dari versi yang sudah stabil melalui branch.
   - **Menjadi cadangan kode** di remote repository seperti GitHub.
2. **git add** memasukkan file atau perubahan ke staging area, yaitu daftar perubahan yang akan direkam. Pada tahap ini belum ada versi yang tersimpan. Sedangkan **git commit** merekam isi staging area ke dalam repository sebagai satu versi permanen yang dilengkapi pesan, nama penulis, dan waktu.
3. Branch baru dapat dibuat dengan perintah berikut:

```
git branch nama-branch         # membuat branch baru
git checkout nama-branch       # pindah ke branch tersebut
git checkout -b nama-branch    # membuat sekaligus pindah
git switch -c nama-branch      # cara singkat pada Git versi
terbaru
```

BAB II

PEMBAHASAN

PRAKTIKUM

1. MEMBUAT AKUN GITHUB
   a. Buka situs GitHub melalui peramban pada alamat `https://github.com/`.
   b. Klik tombol **Sign up** di pojok kanan atas.
   c. Isi email, password, dan username sampai valid, lalu klik **Continue**.
   d. Selesaikan puzzle verifikasi (Protecting your account), kemudian masukkan kode verifikasi yang dikirim GitHub ke email.
   e. Setelah muncul halaman login, masuk menggunakan akun yang baru dibuat, lalu lengkapi profil dengan plan **Free**.
   f. Hasil: akun GitHub berhasil dibuat dengan username **bossmudacalonbosmuda-stack**.

2. INSTALASI GIT
   a. Unduh Git dari `https://git-scm.com/`, lalu install sesuai sistem operasi (Windows) dengan mengikuti petunjuk instalasi sampai selesai.
   b. Buka Visual Studio Code, tekan **Ctrl + `** atau pilih **View > Terminal**, lalu periksa versi Git:

```
git --version
```

   c. Konfigurasi nama pengguna dan email. Email sebaiknya sama dengan email akun GitHub:

```
git config --global user.name "Naufal Nurrochmat Fadhil
Haq"
git config --global user.email "emailanda@gmail.com"
git config --global --list
```

   d. Analisis: perintah `git --version` memastikan Git sudah terpasang, sedangkan konfigurasi `user.name` dan `user.email` dipakai Git sebagai identitas penulis pada setiap commit.

3. MENGELOLA PROYEK DENGAN GIT
   a. Pastikan VS Code membuka folder kerja `Praktikum_Pemrograman_WEB_1/20250910036_NaufalNurrochmat` dan file `Modul_01/Latihan_01_01.html` dari Modul 1 sudah ada.

b. Inisialisasi repository Git baru pada folder proyek melalui terminal VS Code:

```
git init
```

c. Tambahkan file ke staging area:

```
git add Modul_01/Latihan_01_01.html
```

d. Periksa status repository. File `Latihan_01_01.html` akan tampil berwarna hijau pada bagian Changes to be committed, artinya siap di-commit:

```
git status
```

e. Commit perubahan yang ada di staging area:

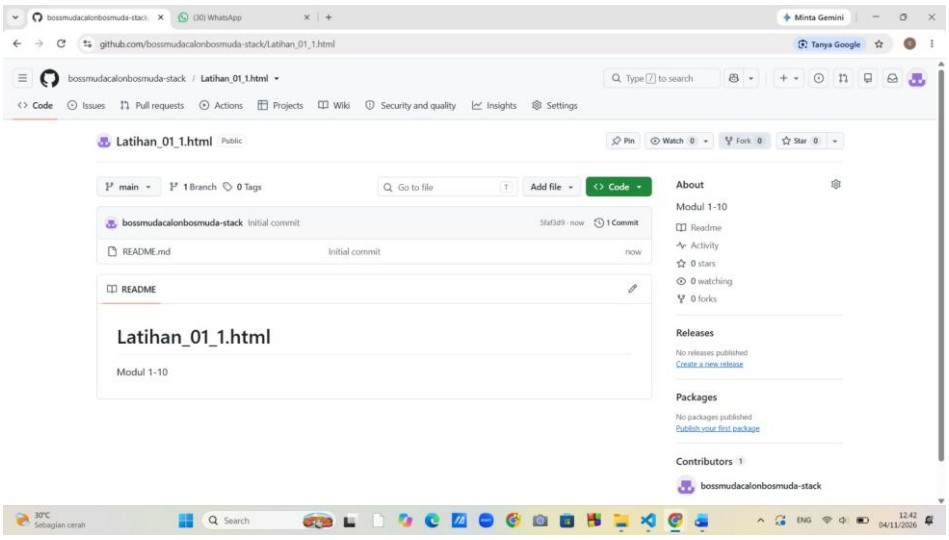
```
git commit -m "Initial commit: menambahkan file
Latihan_01_01.html"
```

f. Catatan: pada modul tertulis `Latihan01_01.html` dan `\git status`. Penulisan yang benar mengikuti Latihan 1 Modul 1, yaitu `Latihan_01_01.html`, dan perintahnya `git status` tanpa tanda `\`.

g. Analisis: `git add` hanya menandai file, sedangkan `git commit` benar-benar menyimpan versi file tersebut ke riwayat repository lokal.

4. MENGHUBUNGKAN KE GITHUB DAN PUSH PERUBAHAN

a. Repository di GitHub sudah dibuat dengan nama Latihan_01_1.html (Public, branch `main`) dan saat ini hanya berisi file `README.md` dari initial commit. Pada modul digunakan nama contoh `pweb1`; pada praktikum ini digunakan repository milik sendiri.



Gambar 7. Repository Latihan_01_1.html di GitHub sebelum push

b. Hubungkan repository lokal ke remote repository GitHub:

```
git remote add origin
https://github.com/bossmudacalonbosmuda-
stack/Latihan_01_1.html.git
```

c. Ubah nama branch utama menjadi `main`:

```
git branch -M main
```

d. Karena repository GitHub sudah memiliki `README.md`, tarik terlebih dahulu isi remote agar riwayat keduanya tergabung. Tanpa langkah ini, `git push` akan ditolak (`rejected ... fetch first`):

```
git pull origin main --allow-unrelated-histories
```

e. Kirim semua commit pada branch `main` ke GitHub:

```
git push -u origin main
```

f. Amati hasilnya dengan me-refresh halaman repository di GitHub.

g. Analisis: setelah push berhasil, folder `Modul_01` berisi `Latihan_01_01.html` tampil bersama `README.md`, jumlah commit bertambah, dan pesan commit terlihat pada riwayat. Proyek kini tersimpan di cloud dan dapat diakses dari komputer mana pun.

BAB III

POSTEST

1. Apa fungsi dari git merge dalam workflow Git?
2. Jelaskan cara mengatasi konflik saat melakukan merge!
3. Tulis langkah-langkah untuk menambahkan file baru ke repository Git dan melakukan commit.

Jawaban:

1. `git merge` menggabungkan perubahan dari satu branch ke branch lain, misalnya hasil pengerjaan fitur pada branch `fitur-css` digabungkan ke `main`, sehingga pekerjaan yang dikembangkan terpisah menyatu dalam satu riwayat.

```
git checkout main
git merge fitur-css
```

2. Konflik terjadi saat dua branch mengubah baris yang sama pada file yang sama. Cara mengatasinya:
   a. Jalankan `git status` untuk melihat file yang berstatus both modified.
   b. Buka file tersebut dan cari penanda konflik `<<<<<<<`, `=======`, dan `>>>>>>>`.
   c. Edit isi file: pilih salah satu versi atau gabungkan keduanya, lalu hapus semua penanda konflik.
   d. Simpan file, kemudian jalankan `git add nama_file`.
   e. Selesaikan merge dengan `git commit` (atau `git merge --continue`).
   f. Kirim hasilnya ke GitHub dengan `git push` bila diperlukan.
3. Langkah menambahkan file baru ke repository dan melakukan commit:

```
# 1. buat file Modul_01/Latihan_01_02.html
git status                  # 2. file untracked
git add Modul_01/Latihan_01_02.html  # 3. stage
git status                  # 4. siap commit
git commit -m "Menambahkan Latihan_01_02.html"  # 5
git push origin main        # 6. kirim ke GitHub
```

BAB IV

TUGAS

1. Buatlah repository Git untuk proyek web dan dokumentasikan proses pembuatan, commit, dan push ke GitHub.

   Dokumentasi proses pada repository **Latihan_01_1.html** (https://github.com/bossmudacalonbosmuda-stack/Latihan_01_1.html):

No	Langkah	Perintah / Aksi	Bukti
1	Buat repository di GitHub	New repository, nama `Latihan_01_1.html`, Public	Gambar
2	Inisialisasi repository lokal	`git init`	Gambar
3	Stage file	`git add Modul_01/Latihan_01_01.html`	Gambar
4	Commit	`git commit -m "Initial commit: ..."`	Gambar
5	Hubungkan ke remote	`git remote add origin <URL>`	Gambar
6	Sinkron dan push	`git pull origin main --allow-unrelated-histories` lalu `git push -u origin main`	Gambar
7	Verifikasi di GitHub	Refresh halaman repository	Gambar

2. Rancanglah workflow Git yang melibatkan branch untuk fitur baru dan integrasi dengan branch utama.

   Alur kerja: branch `main` (versi stabil) → branch `fitur-landing-page` (pengembangan fitur) → digabung kembali ke `main`.

```
git checkout -b fitur-landing-page   # 1. buat branch
# buat/edit file, mis. biodata.html
git add .                            # 2. stage
git commit -m "Menambahkan landing page"   # 3. commit
git push -u origin fitur-landing-page # 4. push branch
git checkout main                    # 5. kembali ke main
git pull origin main                 # 6. ambil update
git merge fitur-landing-page         # 7. gabungkan
git push origin main                 # 8. push hasil
git branch -d fitur-landing-page     # 9. hapus (opsional)
```

Untuk kerja tim, langkah 7 dapat diganti dengan Pull Request di GitHub (Compare & pull request, lalu Merge) agar ada proses review sebelum fitur masuk ke `main`.

3. Simpan seluruh praktikum pada modul sebelumnya ke dalam repository Git yang sudah dibuat.

   Struktur folder yang disimpan di repository:

```
20250910036_NaufalNurrochmat/
├── Modul_01/   (Latihan_01_01 s.d. 01_05, biodata)
└── Modul_02/   (laporan dan dokumentasi Git)
```

   Perintah untuk menyimpan semuanya ke GitHub:

```
git add .
git status
git commit -m "Menambahkan praktikum Modul 1 dan 2"
git push origin main
```

BAB V

KESIMPULAN

Dari praktikum Modul 2 ini, kesimpulannya saya sudah memahami konsep version control dan peran Git dalam mencatat riwayat perubahan proyek. Saya sudah bisa membuat akun GitHub, menginstal dan mengonfigurasi Git, serta menggunakan perintah dasar seperti `git init`, `git add`, `git commit`, `git status`, `git branch`, `git merge`, `git pull`, dan `git push`. Saya juga memahami bahwa `git add` hanya memasukkan perubahan ke staging area, sedangkan `git commit` yang menyimpannya sebagai versi, dan bahwa repository GitHub yang sudah berisi `README.md` perlu di-`pull` terlebih dahulu sebelum di-`push`. Kemampuan ini penting sebagai bekal untuk menyimpan, mengelola, dan berkolaborasi pada proyek desain web di modul-modul selanjutnya.